

Entrega

Proyecto Base de Datos

Agencia de Seguros

Agustín Nicolás König
Comisión 89215

Introducción y Objetivo

Este proyecto busca diseñar una base de datos relacional para una agencia de seguros, con el fin de gestionar clientes, pólizas, siniestros y pagos.

Objetivo: Centralizar la información para reducir redundancia de datos, mejorar la trazabilidad de contratos y optimizar la gestión contable y analítica.

El sistema permitirá el seguimiento de pagos, la vigencia de contratos y la frecuencia de accidentes.

Situación Problemática

Actualmente, la agencia gestiona la información en planillas dispersas, lo que provoca:

Redundancia de datos (clientes duplicados).

Datos huérfanos (siniestros sin póliza válida).

Falta de integridad referencial, que dificulta la liquidación de reclamos y genera retrasos en la toma de decisiones.

Modelo de Negocio

La agencia de seguros ofrece distintos tipos de cobertura (automotor, hogar, salud).

Un cliente puede contratar múltiples pólizas de diferentes tipos.

Cada póliza puede generar pagos periódicos y siniestros asociados.

El sistema busca dar soporte a la gestión integral de estas operaciones, asegurando consistencia y escalabilidad.

Asegurados

- **Tabla: Asegurados**
- **Descripción:** Datos maestros de clientes
- **Campos principales:**
 - *id_asegurado* → Clave primaria, autoincremental
 - *dni* → Documento único
 - *email* → Correo único
 - *nombre, dirección, telefono*
 - *fecha_nacimiento* → útil para cálculo de riesgo

Tipos_Seguro

- **Tabla: Tipos_Seguro**
- **Descripción:** Categorías de seguros ofrecidos
- **Campos principales:**
 - *id_tipo* → Clave primaria, autoincremental
 - *nombre_tipo* → Nombre del tipo de seguro. Por ej., automotor, vida, hogar.
 - *descripción* → detalle textual de la cobertura.

Polizas

- **Tabla: Polizas**

- **Descripción:** Contratos de seguros vinculados a clientes

- **Campos principales:**

- *id_poliza* → Clave primaria, autoincremental.

- *nro_poliza* → Número único de póliza.

- *fecha_inicio* → fecha de inicio del contrato.

- *fecha_fin* → fecha de vencimiento.

- *monto_asegurado* → valor asegurado en la póliza.

- *estado* → estado del contrato (activa o vencida).

- *id_asegurado* → clave foránea hacia asegurados.

- *id_tipo* → clave foránea hacia tipos_seguro.

Siniestros

- **Tabla: Siniestros**

- **Descripción:** Registro de incidentes asociados a pólizas.

- **Campos principales:**

- *id_siniestro* → Clave primaria, autoincremental.

- *fecha_siniestro* → Fecha del incidente.

- *monto_reclamado* → valor reclamado por el asegurado.

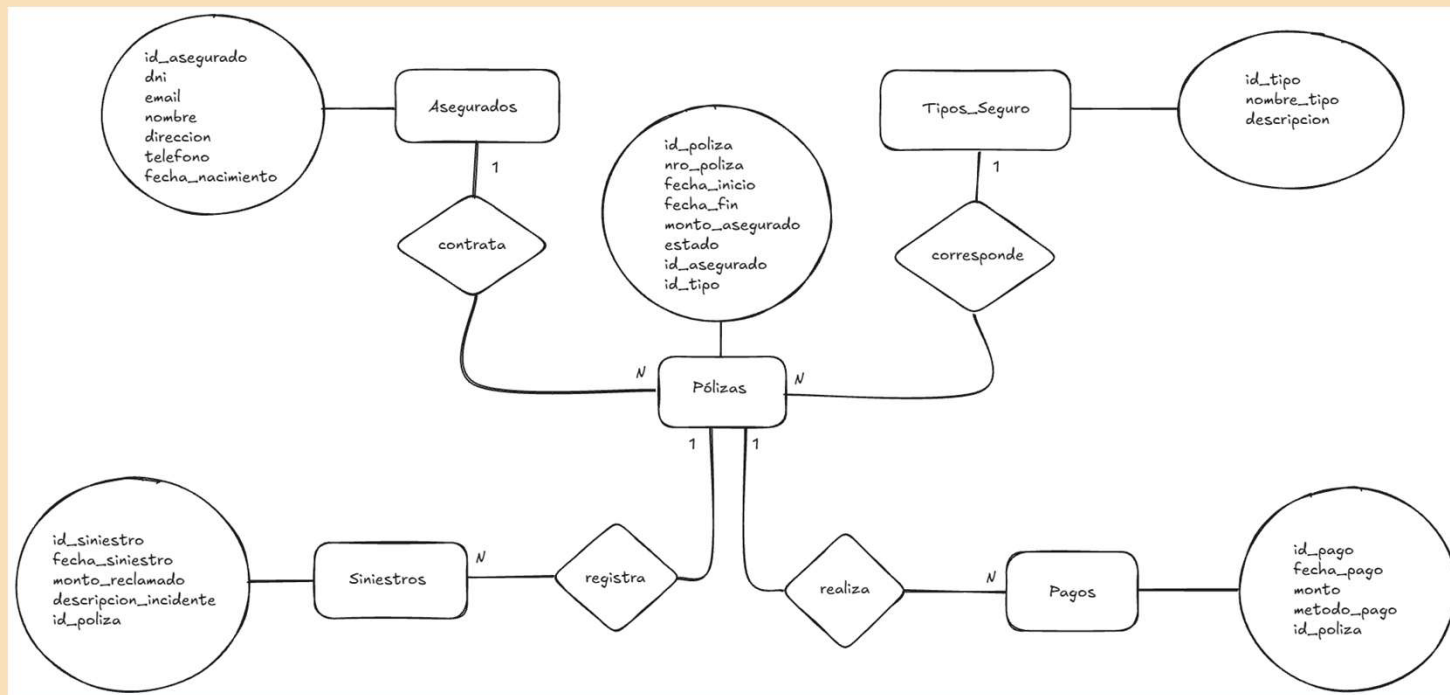
- *descripción_incidente* → detalle del siniestro.

- *id_poliza* → clave foránea hacia pólizas.

Pagos

- **Tabla: Pagos**
- **Descripción:** Registro de pagos de las pólizas.
- **Campos principales:**
 - *id_pago* → Clave primaria, autoincremental.
 - *fecha_pago* → Fecha del pago.
 - *monto* → Importe abonado.
 - *metodo_pago* → método de pago utilizado (tarjeta, transferencia, efectivo).
 - *id_poliza* → clave foránea hacia pólizas.

Diagrama Entidad-Relación



Relaciones

- **Asegurados – Polizas:** Un asegurado puede tener muchas pólizas, pero cada póliza pertenece a un único asegurado **(1 a N)**.
 - **Tipos_Seguro – Polizas:** Un tipo de Seguro puede estar asociado a muchas pólizas, pero cada póliza corresponde a un único tipo **(1 a N)**.
-
- **Polizas – Siniestros:** Una póliza puede registrar multiples siniestros, pero cada siniestro está vinculado a una sola póliza **(1 a N)**.
 - **Polizas – Pagos:** Una póliza puede tener multiples pagos, pero cada pago corresponde a una sola póliza **(1 a N)**.

Entrega 2

14-03-2026

Ajustes de la Entrega N°1

Luego de la primera entrega, se aplicaron las siguientes mejoras de normalización e integridad:

- Se eliminó la restricción de unicidad total sobre el DNI en la tabla *Asegurados* para contemplar excepciones, delegando la identificación unívoca a la clave primaria *id_asegurado*.
- Normalización de los pagos: Creé la tabla dimensional *Metodos_Pago*, transformando el campo de texto en una clave foránea (FK), para reducir la redundancia y asegurar la consistencia de los datos.
- Implementé acciones de DELETE CASCADE en las relaciones clave, garantizando que no existan datos huérfanos, como pagos o siniestros sin una póliza válida.

Listado de Vistas

Vista_Polizas_Asegurados

Objetivo: Simplificar las consultas relacionadas con pólizas y asegurados, permitiendo acceder rápidamente a información relevante como el número de póliza, el nombre del asegurado y el monto asegurado.

Tablas que la componen: Asegurados, Polizas.

Descripción: Esta vista permite visualizar la información de las pólizas junto con los datos del asegurado que la contrató. Facilita consultar qué cliente posee cada póliza sin necesidad de realizar manualmente un JOIN entre tablas.

Vista_Siniestros_Poliza

Objetivo: Facilitar el análisis y seguimiento de los siniestros ocurridos en el sistema, relacionando cada siniestro con su póliza correspondiente.

Tablas que la componen: Polizas, Siniestros.

Descripción: Esta vista muestra los siniestros registrados junto con la póliza asociada.

Permite identificar rápidamente qué póliza tuvo un siniestro y cuál fue el monto reclamado.

Vista_Pagos_Poliza

Objetivo: Facilitar el control de los pagos realizados por los asegurados y su relación con las pólizas contratadas.

Tablas que la componen: Pagos, Polizas

Descripción: Esta vista permite visualizar los pagos realizados por cada póliza, incluyendo la fecha del pago y el monto abonado.

Funciones Personalizadas

total_pagado

Objetivo: Permitir conocer rápidamente cuánto dinero fue abonado por una póliza determinada.

Datos/Tablas: Manipula la tabla Pagos utilizando la función integrada SUM().

Descripción: Esta función calcula el monto total pagado por una póliza sumando todos los pagos registrados en la tabla de pagos asociados a dicha póliza.

cantidad_siniestros

Objetivo: Facilitar el análisis de riesgo y el seguimiento de incidentes asociados a cada póliza.

Datos/Tablas: Consulta la tabla Siniestros.

Descripción: Devuelve el total numérico de registros asociados a una póliza mediante el comando COUNT(*).

Retorna la cantidad total de siniestros registrados para una póliza específica.

Stored Procedures

registrar_pago

Beneficio: Este procedimiento permite registrar un nuevo pago asociado a una póliza, indicando el monto abonado y el método de pago utilizado.

Interacción: Inserta registros en la tabla Pagos recibiendo parámetros de monto, método y póliza.

registrar_siniestro

Beneficio: Automatiza el reporte de incidentes centralizando la descripción y montos reclamados.

Interacción: Realiza una operación INSERT en la tabla Siniestros tras recibir datos dinámicos como parámetros de entrada.

Triggers

validar_pago

Funcionalidad: Actúa como una restricción de integridad de negocio preventivo.

Situación: Se dispara BEFORE INSERT en la tabla Pagos.

Descripción: Este trigger se ejecuta antes de insertar un pago en la tabla Pagos y valida que el monto ingresado sea mayor a cero. Evitar el registro de pagos con montos negativos o inválidos en el sistema.

actualizar_estado_poliza

Funcionalidad: Mantener actualizada la información de las pólizas, indicando que han tenido al menos un siniestro registrado.

Situación: Se dispara AFTER INSERT en la tabla Siniestros.

Descripción: Tras registrarse un siniestro, ejecuta un UPDATE automático en la tabla *Polizas* para cambiar el estado a 'Con siniestro'.

Insertión de Registros

Para la prueba del sistema, se procedió a poblar la base de datos utilizando el sublenguaje DML, mediante la sentencia **INSERT**. Siguiendo los requerimientos de la entrega, se cargaron 10 registros en cada una de las tablas principales, lo que permite realizar pruebas de funcionalidad y verificar que los objetos programables, vistas, funciones y procedimientos, devuelvan los resultados esperados.

El proceso de inserción siguió un orden lógico basado en la jerarquía de las relaciones. Se poblaron primero las tablas primarias (*Asegurados*, *Tipos_Seguro* y *Metodos_Pago*) antes de las tablas secundarias o transaccionales (*Polizas*, *Pagos*, *Siniestros*).

Este enfoque garantiza que cada clave foránea en las tablas secundarias apunte a una clave primaria válida en las tablas principales, evitando la existencia de "datos huérfanos" y asegurando que las relaciones entre tablas permanezcan precisas y sincronizadas.

Script – link a Github

**[https://github.com/konigagus/SistemaSeguros
-KonigAgustin](https://github.com/konigagus/SistemaSeguros-KonigAgustin)**